L02. Combinational Logic (1)

Lecture 02

2.1 조합 논리와 부울 대수

> **조합 논리 = 출력이 현재 입력에만 의존. 식은 부울 대수로 다룸.**

- combinational logic: 출력이 현재 입력에만 의존하는 디지털 시스템.
- (cf.
- 순차 논리) 기술 방법은 진리표, 부울 대수, 논리 게이트 셋.

- 예: tally 회로(1의 개수를 센다), 7-segment 디스플레이, 이진 가산기.

Half adder	입력 A, B, 출력 Carry, Sum. Carry = AB, Sum = A ⊕ B.
Full adder	세 번째 입력으로 carry-in(C_{in})이 붙음. 식은 아래 2.2 에서 유도.

A	B	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

부울 대수는 참/거짓 값 0과 1만 쓰는 대수.

연산	표기
NOT	X′, X̄, ~X
AND	X·Y, X∧Y, X&Y, 또는 그냥 XY
OR	X+Y, X∨Y, X|Y

연산 우선순위

- 우선순위는 NOT → AND → OR. 괄호는 큰 구조를 변수 하나처럼 묶어 줌.

> • 예: $A \cdot B' \neq (AB)'$. $AB' + CD = A(B') + (CD)$ 이지, $(B' + CD)'$ 가 아니다.

- 0과 1은 논리 상태지 이진수가 아님. + 도 부울 OR 이지 이진 덧셈이 아님. 그래서 $1 + 1 = 1$ 이고 $1 + 1 \neq 10$. (1+0 = 1, 0+0 = 0)

진리표에서 식 만들기

- 진리표에서 출력이 1 인 행마다 곱항이 하나씩 나옴. 곱항은 입력이 1인 변수는 그대로, 0인 변수는 보수로 곱해 만듦. 이 곱항을 전부 OR 하면 sum of products(SoP).

예 · Full adder

$C_{out} = A'BC_{in} + AB'C_{in} + ABC_{in}' + ABC_{in} = AC_{in} + BC_{in} + AB$ (부울 대수로 단순화)

$Sum = A'B'C_{in} + A'BC_{in}' + AB'C_{in}' + ABC_{in}$

A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

2.2 부울 대수의 법칙

axioms

- 집합 B 에는 서로 다른 두 원소 $a \neq b$ 가 있음.

closure	$a, b \in B$ 이면 $a+b \in B$, $a \cdot b \in B$.
교환	$a+b = b+a$, $a \cdot b = b \cdot a$.
결합	$(a+b)+c = a+(b+c)$, $(a \cdot b) \cdot c = a \cdot (b \cdot c)$.
항등원	$0, 1 \in B$ 가 있고 $a+0 = a$, $a \cdot 1 = a$.
분배	$a+(b \cdot c) = (a+b) \cdot (a+c)$, $a \cdot (b+c) = (a \cdot b)+(a \cdot c)$.

보수	a+a′ = 1, a·a′ = 0.

기본 정리

표에서 (D) 가 붙은 식은 왼쪽 식의 dual.

법칙	식	쌍대(D)
0, 1 과의 연산	(1) X + 0 = X	(1D) X·1 = X
	(2) X + 1 = 1	(2D) X·0 = 0
idempotent	(3) X + X = X	(3D) X·X = X
involution	(4) (X′)′ = X	
보수	(5) X + X′ = 1	(5D) X·X′ = 0
교환	(6) X+Y = Y+X	(6D) XY = YX
결합	(7) (X+Y)+Z = X+(Y+Z) = X+Y+Z	(7D) (XY)Z = X(YZ) = XYZ
분배	(8) X(Y+Z) = XY + XZ	(8D) X + YZ = (X+Y)(X+Z)
De Morgan	(9) (X+Y)′ = X′Y′	(9D) (XY)′ = X′ + Y′

- 분배법칙의 쌍대 X + YZ = (X+Y)(X+Z) 는 일반 대수에 없는 성질. 증명: (X+Y)(X+Z) = XX + XZ + YX + YZ = X + XZ + XY + YZ = X·1 + XZ + XY + YZ = X(1 + Z + Y) + YZ = X + YZ.

논리적 동등 ≠ 전기적 동등

부울식이 같아도 회로 지연은 다를 수 있음.

교환법칙	X, Y 를 바꿔 넣어도 식은 같음. 하지만 핀 a → y 와 핀 b → y 의 지연이 같다는 보장은 없다.
결합법칙	F = (X+Y)+Z, F = X+(Y+Z), F = X+Y+Z 는 서로 다른 회로도. 게이트를 계단식으로 연결하면 입력 위치에 따라 출력까지 거치는 게이트 수가 달라져 X → F 경로 지연이 서로 다르고, 이 차이를 최적화에 이용함.

De Morgan 의 법칙

DE MORGAN

$(X + Y + Z + \cdots)' = X' \cdot Y' \cdot Z' \cdot \cdots \qquad (X \cdot Y \cdot Z \cdot \cdots)' = X' + Y' + Z' + \cdots$

- 일반형: $F(X_1, \cdots, X_n, 0, 1, +, \cdot)' = F(X_1', \cdots, X_n', 1, 0, \cdot, +)$. 즉 변수는 보수로, 0↔1, +↔· 로 바꾼다.
- AND 식 ↔ OR 식 변환에 씀. 게이트로 보면 AND 의 출력 반전 = 입력 반전된 OR, NOR = 입력 반전된 AND.

Full adder carry-out 을 대수로 줄이기

$C_{out} = A'BC_{in} + AB'C_{in} + ABC_{in}' + ABC_{in}$

① ABC_{in} 을 한 번 더 써 넣는다 (멱등·교환). $A'BC_{in} + ABC_{in} = (A'+A)BC_{in} = BC_{in}$ (분배, 보수, 항등).

② 같은 방식으로 두 번째, 세 번째 항을 반복한다: $AB'C_{in} + ABC_{in} = AC_{in}$, $ABC_{in}' + ABC_{in} = AB$.

$C_{out} = BC_{in} + AC_{in} + AB$

2.3 단순화 정리

아래 법칙을 식에 반복 적용해 최소형을 얻는 방법이 algebraic simplification.

이름	식	쌍대(D)
(1) uniting	$XY + XY' = X$	$(X+Y)(X+Y') = X$
(2) absorption	$X + XY = X$	$X(X+Y) = X$
(3) elimination	$X + X'Y = X + Y$	$X(X'+Y) = XY$
(4) duality	$(X+Y+Z+\cdots)^D = XYZ\cdots$	$(XYZ\cdots)^D = X+Y+Z+\cdots$
(5) multiplying out / factoring	$(X+Y)(X'+Z) = XZ + X'Y$	$XY + X'Z = (X+Z)(X'+Y)$
(6) consensus	$XY + YZ + X'Z = XY + X'Z$	$(X+Y)(Y+Z)(X'+Z) = (X+Y)(X'+Z)$

- (5) 의 증명: $(X+Y)(X'+Z) = XX' + X'Y + XZ + YZ = X'Y + XZ + YZ = X'Y + XZ$ (마지막은 consensus).

Consensus 정리

CONSENSUS

$XY + X'Z + YZ = XY + X'Z \qquad (X+Y)(X'+Z)(Y+Z) = (X+Y)(X'+Z)$

- YZ 는 consensus 항이며 redundant (제거해도 식이 같음).

- 선택한 변수 X(X′)가 꼭 변수 하나일 필요는 없다. 예: AY + BY + A′B′Z 에서 consensus 항 찾기. AY + BY + A′B′Z = (A+B)Y + A′B′Z = (A+B)Y + (A+B)′Z = (A+B)Y + (A+B)′Z + YZ.
- **쓰임**: 논리 최소화 중에 씀. consensus 항을 추가하고 항을 합치는 과정을 반복해 식을 더 간단히 만들 수 있음. 하지만 가능한 모든 식 중 항이나 리터럴 수가 가장 적은 식을 반드시 찾는 것은 아님. 추가한 항으로 다음 단순화가 가능해짐.

예 · 확장 후 축소

증명할 식: A′BD′ + BCD + ABC′ + AB′D = BC′D′ + AD + A′BC

① **확장**: 서로 consensus 인 항을 추가.

- A′BD′ 와 ABC′ 의 consensus 는 BC′D′
- A′BD′ 와 BCD 의 consensus 는 A′BC
- BCD 와 ABC′ 의 consensus 는 ABD

좌변 = A′BD′ + BCD + ABC′ + AB′D + BC′D′ + A′BC + ABD

② **축소**: AB′D + ABD = AD 로 묶음.

좌변 = A′BD′ + BCD + ABC′ + (AD + BC′D′ + A′BC)

③ **다시 확장 후 삭제**: 괄호 안의 항으로 consensus 를 만든 뒤 지움.

- BC′D′ 와 AD 의 consensus 는 ABC′
- AD 와 A′BC 의 consensus 는 BCD
- BC′D′ 와 A′BC 의 consensus 는 A′BD′

이 세 항이 모두 괄호 안 항들의 consensus 이므로 지움.

= AD + BC′D′ + A′BC

---

duality

- 쌍대 식: 식의 0↔1, AND↔OR 를 맞바꾼 식.
- 변수는 그대로.
- $F^D(X_1, \cdots, X_n, 0, 1, +, \cdot) = F(X_1, \cdots, X_n, 1, 0, \cdot, +)$

- 쌍대 식은 원래 식과 같은 함수일 수도, 다른 함수일 수도 있음. 같은 경우: F = X + 0 = X 의 쌍대는 F^D = X·1 = X. 다른 경우: F = X + Y 의 쌍대는 F^D = XY.
- **쌍대 원리**: 모든 부울식에 쌍대가 있으며 참인 부울 등식의 쌍대 등식도 참. AND, OR, NOT 으로 쓴 식은 AND 와 OR 를 맞바꾼 쌍대 항등식을 가짐.

> • 예: (원식) 1 AND 0 = 0 → (쌍대) 0 OR 1 = 1.

2.4 논리 게이트

입력이 2개인 함수는 몇 개일까? 입력 조합은 4가지(00, 01, 10, 11), 각 조합의 출력은 0 또는 1. 그래서 $2^4 = 16$개.

X	Y	F0	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	F11	F12	F13	F14	F15
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

함수	이름	식
F0	상수 0	0
F1	AND	X·Y
F2		X·Y′
F3		X
F4		X′·Y
F5		Y
F6	XOR	X ⊕ Y
F7	OR	X + Y
F8	NOR	(X+Y)′
F9	XNOR	X ≡ Y
F10		Y′
F11		X + Y′
F12		X′
F13		X′ + Y
F14	NAND	(XY)′
F15	상수 1	1

XOR 와 XNOR

- **XOR**: X ⊕ Y = 1 이면 (X=0 이고 Y=1) 또는 (X=1 이고 Y=0). Z = X ⊕ Y = X′Y + XY′.
- **XNOR**(equivalence gate): Z = (X ⊕ Y)′ = X′Y′ + XY. 증명: (XY′ + X′Y)′ = (XY′)′(X′Y)′ = (X′+Y)(X+Y′) = X′Y′ + XY (multiplying out).

	XOR (⊕)	XNOR (≡)
(1)	X ⊕ 0 = X	X ≡ 0 = X′
(2)	X ⊕ 1 = X′	X ≡ 1 = X
(3)	X ⊕ X = 0	X ≡ X = 1
(4)	X ⊕ X′ = 1	X ≡ X′ = 0
(5) 교환	X ⊕ Y = Y ⊕ X	X ≡ Y = Y ≡ X
(6) 결합	(X⊕Y)⊕Z = X⊕(Y⊕Z) = X⊕Y⊕Z	(X≡Y)≡Z = X≡(Y≡Z)
(7) 분배	X(Y ⊕ Z) = XY ⊕ XZ	AND 는 XNOR 에 분배되지 않음. X = 0 이면 왼쪽은 0, XY ≡ XZ 는 1.
(8)	(X⊕Y)′ = X⊕Y′ = X′⊕Y = XY + X′Y′ = X≡Y	(X≡Y)′ = X≡Y′ = X′≡Y = X′Y + XY′ = X⊕Y

- XOR 입력 한 곳의 bubble 은 출력 쪽으로 옮겨도 같음. bubble 하나는 XOR ↔ XNOR 를 바꾸고, bubble 두 개는 서로 상쇄 (bubble deletion).

예 · A′ ⊕ B ⊕ C 를 전개

A′ ⊕ B ⊕ C = [A′B′ + (A′)′B] ⊕ C = (A′B′ + AB) ⊕ C

= (A′B′ + AB)C′ + (A′B′ + AB)′C = (A′B′ + AB)C′ + (A′B + AB′)C

= A′B′C′ + ABC′ + A′BC + AB′C